

LEVEL 1-ABSTRACTION

A message generator is built using abstraction, but the implementation is incomplete.

The system:

1. Uses an abstract base to enforce a contract.
2. Stores a hidden riddle in a structured form.
3. Applies selective filtering before revealing the message.

Your task:

- Complete the abstract contract.
- Implement the concrete class correctly.
- Ensure the correct method is executed.
- The output is a riddle. The answer is NOT printed.

Warning:

- If your filtering condition is wrong, the clue becomes unreadable.
- Choosing the wrong implementation will still compile but produce useless output.

```
abstract class PuzzleCore {

protected String[] data;

// SECTION 1 – Contract
// FILL 1: define abstract behavior
public abstract String _____();
}

class RiddleUnit extends PuzzleCore {

public RiddleUnit() {
    data = new String[]{
        "I",
        "have",
        "a",
        "neck",
        "but",
        "no",
        "head",
        "and",
        "two",
        "arms",
        "but",
        "no",
        "hands"
    };
}
```

```

// SECTION 2 – Implementation
// FILL 2: override annotation
@_____

// FILL 3: correct signature
public _____ generate() {

    String output = "";

    for(int i = 0; i < data.length; i++) {

        // FILL 4: include only words longer than 2 letters
        if(_____) {
            output += data[i] + " ";
        }
    }

    return output;
}
}

public class Level1 {

    public static void main(String[] args) {

        // SECTION 3 – Trigger abstraction

        // FILL 5: correct object creation
        PuzzleCore p = new _____();

        // FILL 6: call correct method
        String clue = p._____;

        System.out.println("LEVEL 1 UNLOCKED");
        System.out.println(clue);
        System.out.println("Solve the riddle.");
    }
}

```

FILL 1 → generate FILL 2 → Override

FILL 3 → String

FILL 4 → data[i].length() > 2 FILL 5 → RiddleUnit

FILL 6 → generate

LEVEL 1 UNLOCKED

have neck but head and two arms but hands

Solve the riddle.

PLAYER THINKING

Original words:

I have a neck but no head and two arms but no hands

After filtering, they must mentally reconstruct the riddle.

👉 The answer: SHIRT

LEVEL 2: ENCAPSULATION

A secret word is hidden inside a data vault.

The vault does not store the original message directly.

Instead, the message is transformed before being stored, and what is printed is not the original word.

Your task:

1. Study the given code carefully.
2. Complete the missing parts related to encapsulation.
3. Understand how the message is transformed before and after storage.
4. The output will show a scrambled word.
5. Unscramble it to find the original computer-science term.

Hint:

The original word is a fundamental programming concept.

```
public class Level2 {  
  
    public static void main(String[] args) {  
  
        LabSecret obj = new LabSecret();  
  
        int[] data = {  
            82,69,67,85,82,83,73,79,78    // hidden word  
        };  
  
        String text = "";  
  
        for(int i = 0; i < data.length; i++) {  
            text = text + (char)data[i];  
        }  
    }  
}
```

```

char[] arr = text.toCharArray();

// Transformation 1: swap neighbours
for(int i = 0; i < arr.length - 1; i += 2) {
    char temp = arr[i];
    arr[i] = arr[i + 1];
    arr[i + 1] = temp;
}

// Transformation 2: rotate right by 2
for(int r = 0; r < 2; r++) {
    char last = arr[arr.length - 1];
    for(int i = arr.length - 1; i > 0; i--) {
        arr[i] = arr[i - 1];
    }
    arr[0] = last;
}

// FILL 1
obj._____(new String(arr));

System.out.println("LEVEL 2 UNLOCKED");

// FILL 2
System.out.println("JUMBLED WORD: " + obj._____());

System.out.println("Unscramble to get the real word.");
}

}

class LabSecret {

private String clue;

// FILL 3
public void _____(String clue) {
    this.clue = clue;
}

// FILL 4
public String _____() {
    return clue;
}
}

```

✓ CORRECT FILLS (ADMIN)

FILL 1 → setClue

FILL 2 → getClue

FILL 3 → setClue

FILL 4 → getClue

EXPECTED OUTPUT (WHAT PLAYERS SEE)

LEVEL 2 UNLOCKED

JUMBLED WORD: ONRSUEIRC

Unscramble to get the real word.

PLAYER THINKING

- The vault hides the word
- The getter reveals only the scrambled form
- Undo transformations mentally
- Use the hint

 Correct answer to submit: RECURSION

LEVEL 3 — INHERITANCE

A decoder system is built using inheritance.

Rules:

1. A base class provides core behavior.
2. An intermediate class modifies the data.
3. A final class applies the last transformation.

However:

- The correct class must be used.
- Some transformations are misleading.
- Only one execution path produces the correct clue.

Tasks:

- Complete the inheritance chain.
- Fill the missing logic.
- Trigger the correct class.
- The output is a scrambled word.

- Unscramble it to find the correct CS term.

Note:

Only one word is the correct answer. public class Level3 {

```
public static void main(String[] args) {
```

```
    // FILL 5: correct object type
    Decoder obj = new _____();
```

```
    obj.decode();
```

```
}
```

```
// Base class class Decoder {
```

```
protected String process(String input) {
    return input;
}
```

```
void decode() {
    System.out.println("Invalid path");
}
}
```

```
// Intermediate class class ShiftDecoder extends Decoder {
```

```
// FILL 1
```

```
@_____
```

```
protected String process(String input) {
```

```
    char[] arr = input.toCharArray();
```

```
    // shift left by one
```

```
    char first = arr[0];
```

```
    for(int i = 0; i < arr.length - 1; i++) {
        arr[i] = arr[i + 1];
```

```
    }
```

```
    arr[arr.length - 1] = first;
```

```
    return new String(arr);
```

```
}
```

```
// Final class // FILL 2 class FinalDecoder _____ ShiftDecoder {
```

```
@Override
```

```
void decode() {
```

```

System.out.println("LEVEL 3 UNLOCKED");

int[] data = {
    83,84,65,67,75    // STACK
};

String text = "";

for(int i = 0; i < data.length; i++) {
    text += (char)data[i];
}

String mid = process(text);

char[] arr = mid.toCharArray();

// FILL 3: reverse string
for(int i = 0; i < arr.length / 2; i++) {
    char t = arr[i];
    arr[i] = arr[_____];
    arr[_____] = t;
}

// FILL 4: print result
System.out.println("SCRAMBLED: " + new String(arr));
System.out.println("Unscramble to get the CS term.");
}

}

```

✓ CORRECT FILLS (ADMIN)

FILL 1 → Override
 FILL 2 → extends
 FILL 3 → arr.length - 1 - i
 FILL 4 → arr.length - 1 - i
 FILL 5 → FinalDecoder

EXPECTED OUTPUT

```

LEVEL 3 UNLOCKED
SCRAMBLED: KCATS
Unscramble to get the CS term.

```

PLAYER THINKING

- Trace inheritance chain
- Understand overridden process
- Follow shift + reverse
- Unscramble:

👉 STACK

LEVEL 4 — POLYMORPHISM

A polymorphic decoder hides a single tech-related word.

Rules:

1. A base class provides a default behavior.
2. Multiple subclasses exist, but only one correctly reveals the clue.
3. The revealed output is transformed multiple times and is not the answer.
4. You must:
 - choose the correct class
 - ensure overriding is correct
 - understand the transformations

The final printed word is scrambled.

Unscramble it to get the real answer.

```
public class Level4 {  
  
    public static void main(String[] args) {  
  
        // FILL 6 – choose correct runtime class  
        ClueBase obj = new _____();  
    }  
}
```



```

        obj.showClue();
    }

}

class ClueBase {

    void showClue() {
        System.out.println("WRONG PATH");
    }

    String scramble(String s) {
        return s;
    }

}

// Trap class (compiles but wrong) class FakeClue extends ClueBase {

    void showClue() {
        System.out.println("LEVEL 4 UNLOCKED");
        System.out.println("DECODE: RANDOMTEXT");
    }

}

// Real class // FILL 1 class FinalClue _____ ClueBase {

// FILL 2
@_____

// FILL 3
_____ showClue() {

    System.out.println("LEVEL 4 UNLOCKED");

    int[] data = {
        65,76,71,79,82,73,84,72,77
    };

    String text = "";

    for(int i = 0; i < data.length; i++) {
        text += (char)data[i];
    }

    String mid = transform(text);

    String finalOut = scramble(mid);

```

```

        System.out.println("DECODE: " + finalOut);
        System.out.println("(Hint: Core CS concept)");
    }

    String transform(String input) {

        char[] arr = input.toCharArray();

        // swap neighbors
        for(int i = 0; i < arr.length - 1; i += 2) {
            char t = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = t;
        }

        return new String(arr);
    }

    // FILL 4 – override
    @Override
    String scramble(String s) {

        char[] arr = s.toCharArray();

        // reverse
        for(int i = 0; i < arr.length / 2; i++) {
            char t = arr[i];
            arr[i] = arr[arr.length - 1 - i];
            arr[arr.length - 1 - i] = t;
        }

        // rotate left by one
        char first = arr[0];
        for(int i = 0; i < arr.length - 1; i++) {
            arr[i] = arr[i + 1];
        }
        arr[arr.length - 1] = first;

        return new String(arr);
    }

}

```

✓ CORRECT FILLS (ADMIN)

FILL 1 → extends
 FILL 2 → Override
 FILL 3 → void

FILL 4 → Override
FILL 6 → FinalClue

EXPECTED OUTPUT

LEVEL 4 UNLOCKED
DECODE: MHORTILGA
(Hint: Core CS concept)

PLAYER THINKING

Reverse transformations:

MHORTILGA → ALGORITHM

👉 Final Answer: ALGORITHM